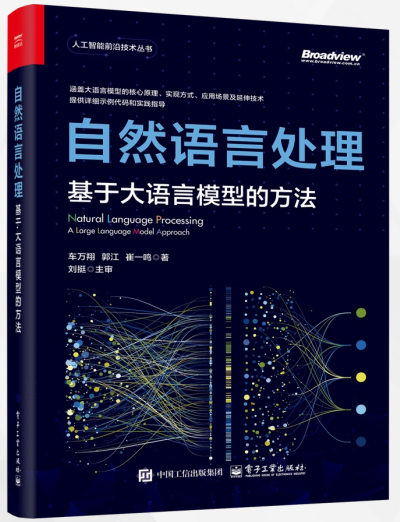


语言模型

车万翔

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

本讲内容与学习目标

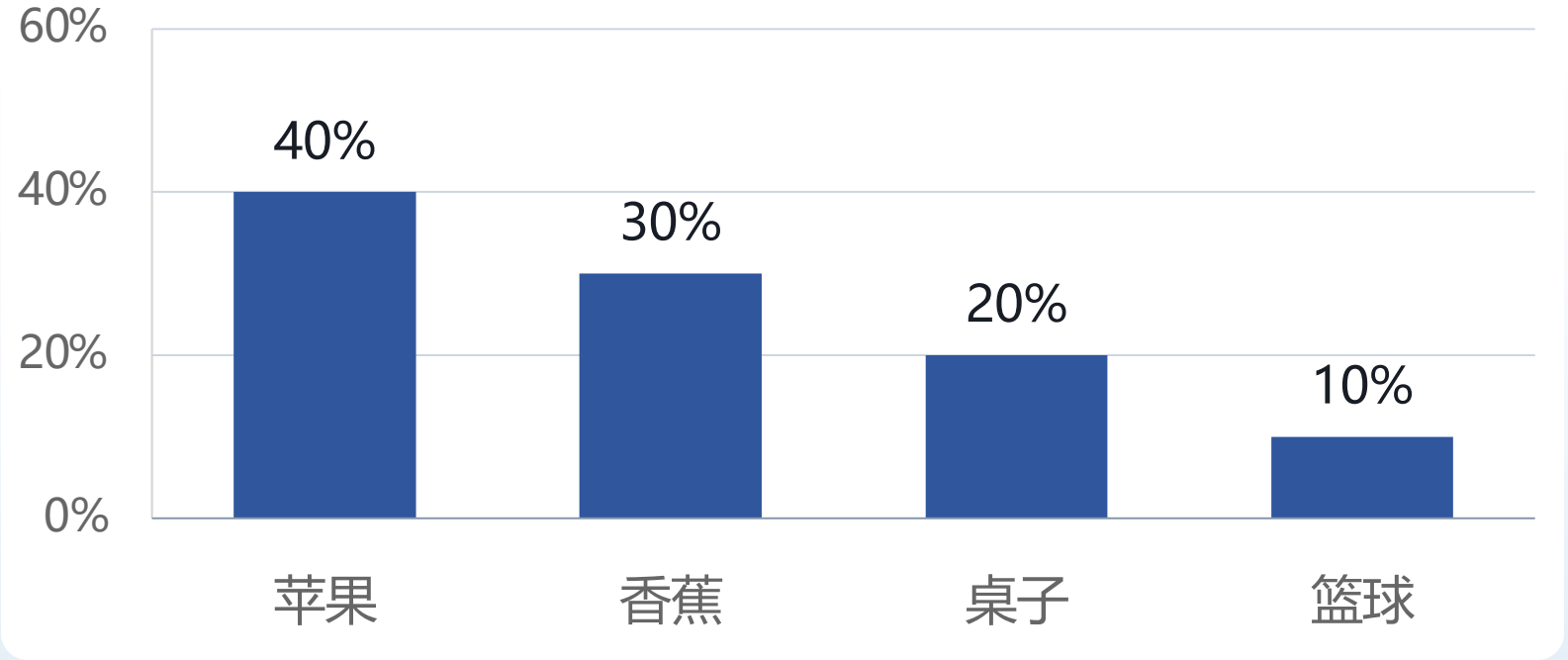
课堂内容	学完后能够
概念与N元模型	手算句子概率与平滑概率
神经语言模型	解释输入、输出与因果Mask
训练与文本生成	区分训练目标和解码策略
困惑度与评价	正确累计PPL，判断能否比较
研究拓展与练习	联系新方法，检查常见错误

语言模型

- 语言模型（Language Model，LM）
  - 描述一段自然语言的概率或给定上文时下一个词出现的概率
    - $P(w_1, \dots, w_l)$，$P(w_{l+1}|w_1, \dots, w_l)$
  - 以上两种定义等价（链式法则）
    - $P(w_1, \dots, w_l) = P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \dots P(w_l|w_1w_2 \dots w_{l-1}) = \prod_{i=1}^{l} P(w_i|w_{1:i-1})$
- 广泛应用于多种自然语言处理任务
  - 机器翻译（词排序）
    - P(the cat is small) > P(small the is cat)
  - 语音识别（词选择）
    - P(there are **four** cats) > P(there are **for** cats)

下一个词的概率分布

“他 喜欢 吃 ___”



概率总和为1

模型给出分布
解码策略选词

语言模型的概率评分与文本生成

概率评分

比较候选表达的可能性
the cat is small / small the is cat

文本生成

给定上文，计算下一词分布
选择一个词，再把它加入上文

应用场景

输入法补全、语音识别、机器翻译和文本续写

链式法则：句子概率的分解

$$P(w_1 \ldots w_n, \text{EOS}) = \prod_t P(w_t \mid w_1 \ldots w_{t-1})$$

约定 w_{n+1} = EOS，乘积取 t = 1…n+1。

已有上文	待预测词元	给定概率
<bos>	我	0.50
<bos> 我	喜欢	0.40
<bos> 我 喜欢	读书	0.25
<bos> 我 喜欢 读书	<eos>	0.80

P(我 喜欢 读书 <eos>) = 0.04

词元与特殊符号

词元	用途
<pad>	批次补齐，排除其目标损失
<unk>	表示词表之外的输入
<bos>	提供生成的起点
<eos>	表示句子结束

“词元”可以是字、词或子词。切分方式决定预测单位。

N元模型与历史窗口

预测“我 今天 很 喜欢 ___”中的下一个词

模型	保留的历史	条件概率
Unigram，一元	不看上文	$P(w_t)$
Bigram，二元	喜欢	$P(w_t \mid w_{t-1})$
Trigram，三元	很 喜欢	$P(w_t \mid w_{t-2}, w_{t-1})$

N元模型保留前 N−1 个词（该假设被称为**马尔可夫假设**（Markov Assumption））
增大N会增加数据稀疏和存储压力。

计数估计：在相同历史下比较后继词

历史 h = “我 喜欢”，共出现4次

后继词	C(h, w)	最大似然概率
读书	3	3 / 4 = 0.75
音乐	1	1 / 4 = 0.25
运动	0	0
<eos>、<unk>	各0	各0

$P(w \mid h) = C(h, w) / C(h)$

教学计数。分母统计历史作为前缀的次数，$C(h) = \Sigma_w C(h, w)$。

零概率与数据稀疏

见过词，未见组合　　词表里有“运动”，但训练语料没有“我 喜欢 运动”

未见整个历史　　若 C(h) = 0，计数比值无法直接计算

未登录词 OOV　　词不在词表，需要<unk>映射或改用子词

一句话中只要有一项概率为0，连乘结果就是0。

加δ平滑：给未见事件分配概率

$$P\delta(w \mid h) = [C(h, w) + \delta] / [C(h) + \delta V]$$

候选词	计数	δ = 0	δ = 1	δ = 0.5
读书	3	0.750	4 / 9	7 / 13
音乐	1	0.250	2 / 9	3 / 13
运动	0	0	1 / 9	1 / 13
<eos>	0	0	1 / 9	1 / 13
<unk>	0	0	1 / 9	1 / 13

此处 V = 5，C(h) = 4。δ > 0，通过验证集选择其大小。

Python实现：计数与平滑概率

```
from collections import Counter
h = ("我", "喜欢")
observed = ["读书", "读书", "读书", "音乐"]
counts = Counter((h, word) for word in observed)
vocab = ["读书", "音乐", "运动", "<eos>", "<unk>"]
delta = 0.5
total = sum(counts[h, w] for w in vocab)
probs = [(counts[h, w] + delta) /
         (total + delta * len(vocab)) for w in vocab]
print(sum(probs), probs[2])  # 约1.0，0.0769
```

预测词表需事先固定。遇到未登录词时先映射为<unk>。

N元模型的生成过程

步数	当前历史（两个词）	采样得到
1	<bos> <bos>	我
2	<bos> 我	喜欢
3	我 喜欢	读书
4	喜欢 读书	<eos>，停止

统计概率后，也可以生成训练语料中没有的完整句子。

这是一条教学示意路径。每一步重新查找当前历史对应的条件分布。

原始文本“我 喜欢 读书”本身就包含训练目标

输入历史	正确的下一个词	分类空间
<bos> <bos>	我	整个词表
<bos> 我	喜欢	整个词表
我 喜欢	读书	整个词表
喜欢 读书	<eos>	整个词表

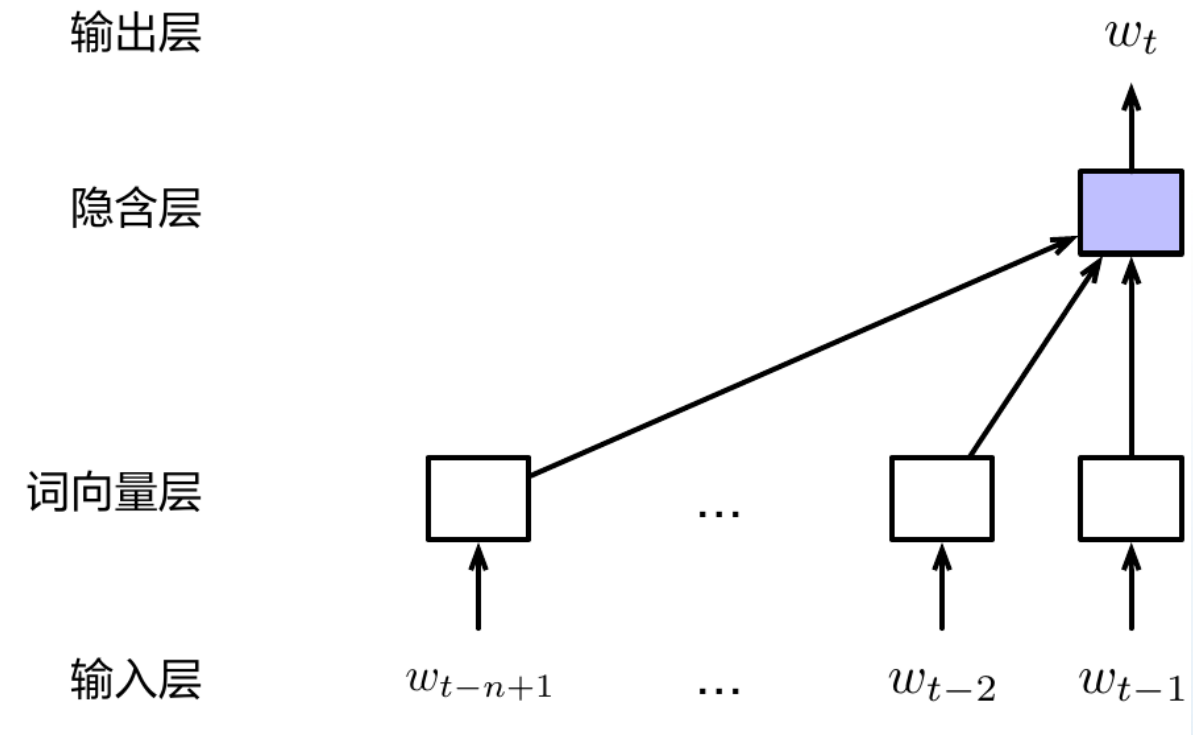
模型学习在词表上的分类。监督信号由文本中的相邻关系自动构造。

前馈神经网络语言模型

词向量 每个ID查表

拼接 按词序排列向量

MLP 输出词表上的得分



FFNNLM用神经网络估计固定历史窗口的条件概率。

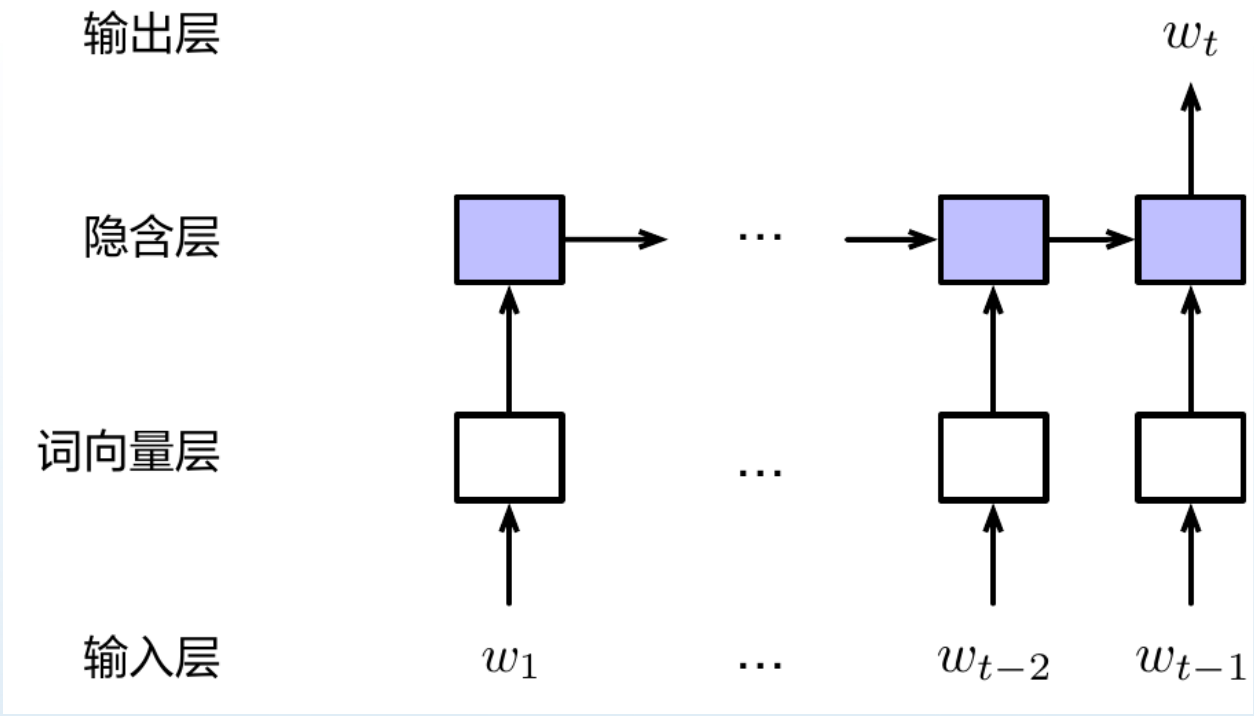
PyTorch实现：固定窗口的FFNNLM

```
import math
import torch
from torch import nn
import torch.nn.functional as F
ffnn = nn.Sequential(
    nn.Embedding(12, 8), nn.Flatten(start_dim=1),
    nn.Linear(3 * 8, 16), nn.ReLU(), nn.Linear(16, 12))
ids = torch.tensor([[2, 4, 5], [4, 5, 6]])
logits = ffnn(ids)
print(logits.shape)  # torch.Size([2, 12])
```

保留logits用于交叉熵。需要采样时，再对logits调用Softmax。

循环神经网络语言模型

“他 感冒 了，于是 下班 之后 去 了 ___”



递推状态
汇总已读上文

参数跨时刻共享
序列变长时
仍共享这组参数

RNN / LSTM可以读取变长上文，但有限维状态不保证记住所有历史细节。

PyTorch实现：LSTM逐位置预测

```
class TinyLSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.emb = nn.Embedding(12, 8)
        self.rnn = nn.LSTM(8, 8, batch_first=True)
        self.head = nn.Linear(8, 12)
    def forward(self, ids):
        h, _ = self.rnn(self.emb(ids))
        return self.head(h)
model = TinyLSTM()
logits = model(ids)
print(logits.shape)  # [2, 3, 12]
```

输出形状为[B, T, V]，保留每个位置的词表得分，不对句子做平均池化。

教师强制与输入目标错位

位置	0	1	2	3
输入	<bos>	我	喜欢	读书
目标	我	喜欢	读书	<eos>

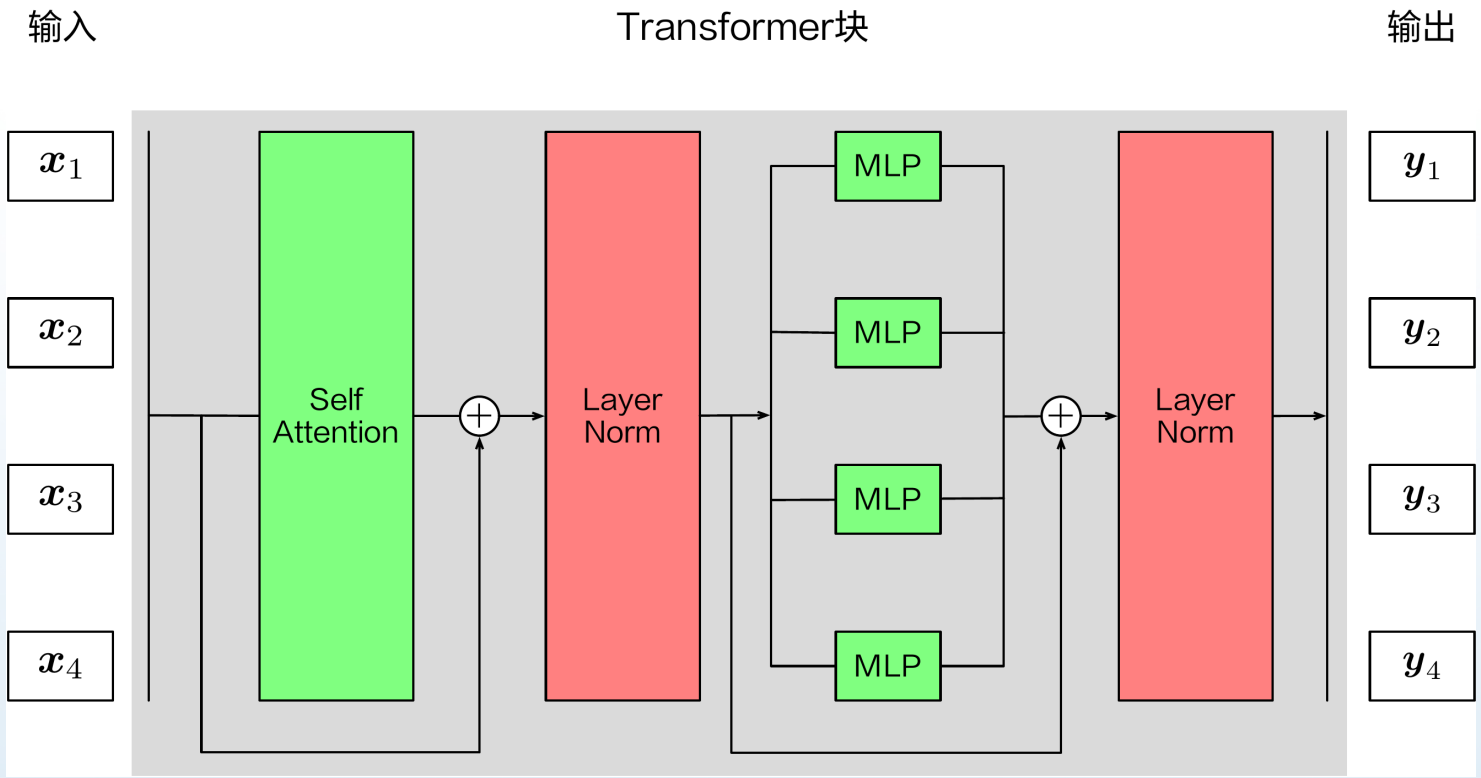
训练时　　输入真实上文，学习预测下一词

生成时　　将已经生成的词接回输入，继续预测

正确错位与因果Mask共同防止“提前看到答案”。

Transformer语言模型的计算过程

词向量与位置表示相加，再通过因果Transformer块。



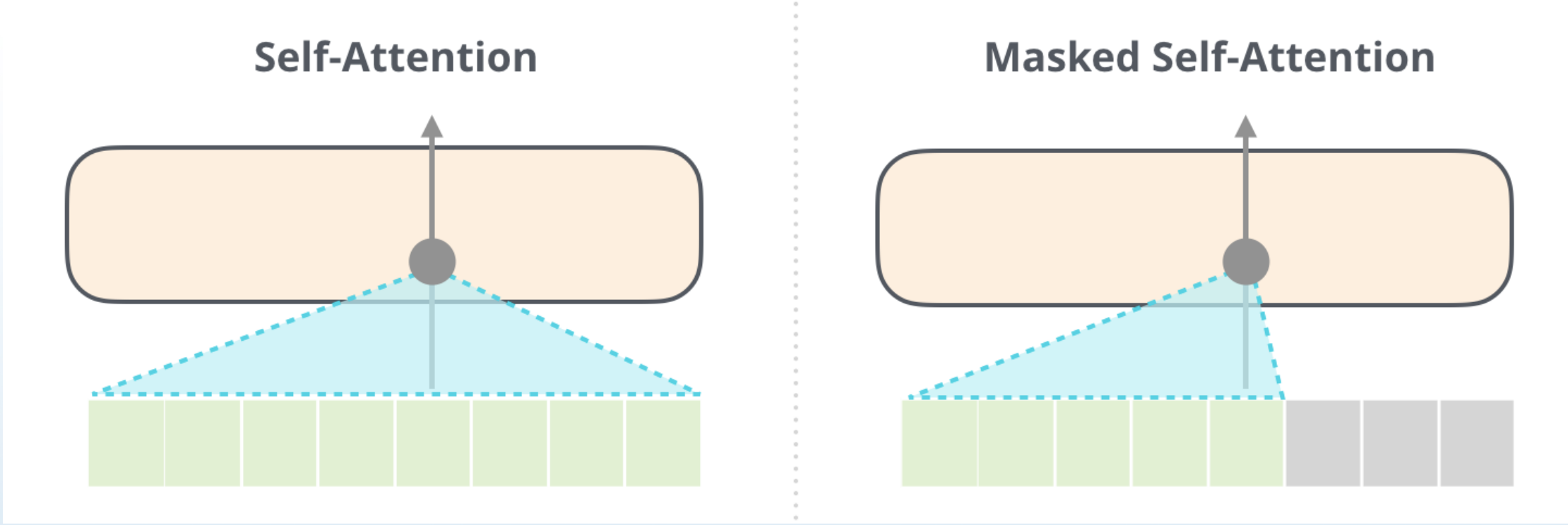
因果注意力
只读取自身与过去

词表输出层
将D维表示映射为V个logits

图中Self-Attention设为因果自注意力。输出表示还需映射到词表。

因果注意力与下一词预测

预测下一词时，可以读取当前输入词与更早的词



因果Mask屏蔽未来位置。Padding需要另外处理。

因果Mask：允许读取的下三角区域

当前输入	<bos>	我	喜欢	读书
<bos>	1	0	0	0
我	1	1	0	0
喜欢	1	1	1	0
读书	1	1	1	1

1：可以读取
0：屏蔽

“我”所在行
用来预测“喜欢”

```
blocked = torch.ones(4, 4, dtype=torch.bool).triu(1)
scores = torch.zeros(4, 4).masked_fill(blocked, -torch.inf)
weights = F.softmax(scores, dim=-1)
```

此代码的blocked中True表示屏蔽。每一行Softmax结果仍然归一化为1。

PyTorch实现：因果编码与词表输出

```
tok, pos = nn.Embedding(12, 8), nn.Embedding(8, 8)
block = nn.TransformerEncoderLayer(
    8, 2, 32, dropout=0.0, batch_first=True)
head = nn.Linear(8, 12)
T = ids.size(1)
mask = torch.ones(T, T, dtype=torch.bool).triu(1)
x = tok(ids) + pos(torch.arange(T))
h = block(x, src_mask=mask,
          src_key_padding_mask=ids.eq(0))
logits = head(h)  # [B, T, V]
```

同时指定位置表示、因果Mask和Padding Mask。输出层预测下一词的logits。

四类语言模型的比较

模型	如何使用历史	主要特点	主要限制
N元模型	统计最近N−1词	可解释、无需反向传播	稀疏与未见组合
FFNNLM	拼接固定窗口词向量	共享词向量与网络参数	历史窗口固定
RNN / LSTM	递推更新隐状态	支持变长历史	时间递推、有限记忆
Transformer	窗口内的因果注意力	各位置训练计算可并行	窗口与注意力开销

四类模型都可以估计下一词概率。具体效果取决于数据、训练和评估设置。

训练数据与评价数据的划分

1	**划分原始文档**	训练集、验证集、测试集互不重叠
2	**训练词表**	只使用训练集，固定特殊词元ID
3	**构造输入目标**	按窗口截取，或将句子整体错开一位
4	**组织批次**	右侧Padding，记录哪些目标有效
5	**选择与评价**	验证集调参，最后在测试集计算PPL

先划分文档，再切训练窗口，避免重叠片段进入训练集和测试集。

语言模型的交叉熵损失

每个有效位置都是一次“词表分类”

张量	形状	说明
logits	[B, T, V]	每个位置V个原始得分
targets	[B, T]	下一词的ID，long类型

```
ids = torch.tensor([[2, 4, 5, 6], [2, 4, 7, 0]])
targets = torch.tensor([[4, 5, 6, 3], [4, 7, 3, 0]])
logits = model(ids)
loss = F.cross_entropy(logits.reshape(-1, 12),
                       targets.reshape(-1), ignore_index=0)
```

本例7个有效目标，包含EOS。CrossEntropyLoss前面保留原始logits。

PyTorch训练：损失、梯度与参数更新

```
opt = torch.optim.Adam(model.parameters(), lr=0.01)
model.train()
for step in range(100):
    opt.zero_grad(set_to_none=True)
    logits = model(ids)
    loss = F.cross_entropy(logits.reshape(-1, 12),
                           targets.reshape(-1), ignore_index=0)
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    opt.step()
```

示例在一个小批次上训练，用于理解流程。真实实验需要独立验证集和测试集。

自回归文本生成的循环

```
model.eval()
x = torch.tensor([[2]])  # 从BOS开始，单样本
with torch.inference_mode():
    for _ in range(12):
        z = model(x)[:, -1, :]
        z[:, [0, 2]] = -torch.inf
        nxt = torch.multinomial(F.softmax(z, -1), 1)
        x = torch.cat((x, nxt), dim=1)
        if nxt.item() == 3:  # EOS
            break
```

预测分布、抽取词元、加入上文，重复至EOS或最大新词元数。

贪心解码与随机采样

候选词	苹果	香蕉	桌子	篮球
概率	0.4	0.3	0.2	0.1

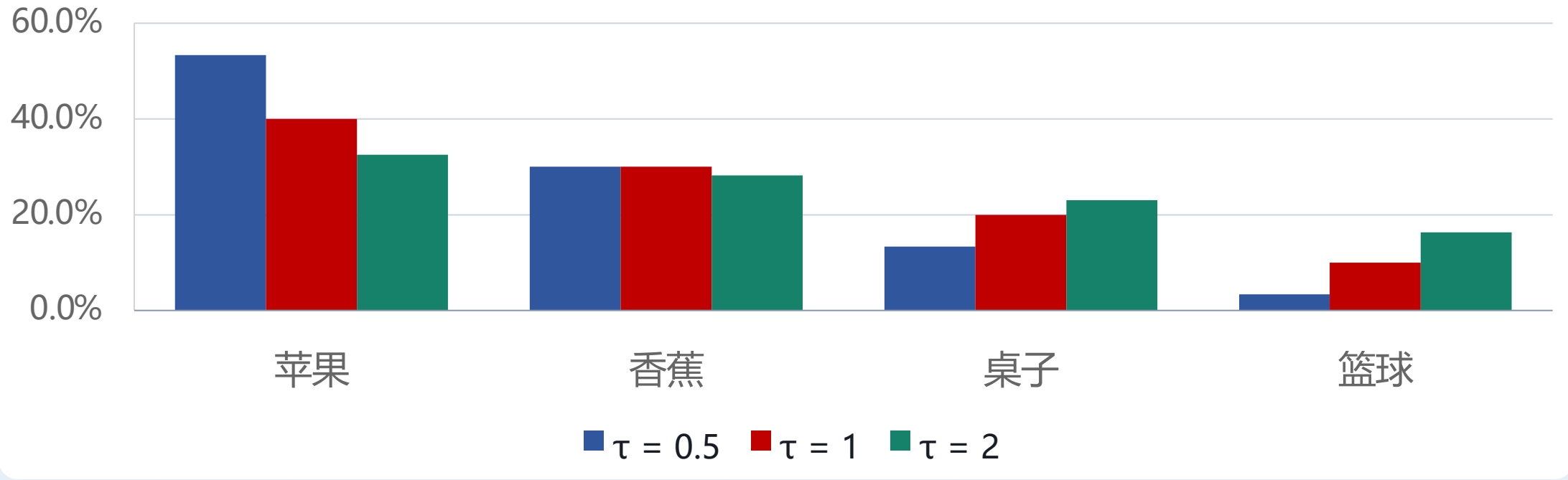
贪心解码　　选择概率最大的“苹果”

随机采样　　按概率抽取，四个候选都有机会

```
z = torch.tensor([0.4, 0.3, 0.2, 0.1]).log()
greedy_id = z.argmax()
sampled_id = torch.multinomial(F.softmax(z, -1), 1)
```

温度：调整概率分布的集中程度

$$p_i(\tau) = \exp(z_i / \tau) / \Sigma_j \exp(z_j / \tau),\ \tau > 0$$



τ较小时更集中，τ较大时更分散。排序不变，概率大小会改变。

Top-k与Top-p：选择采样候选集合

候选（已降序）	原概率	累计概率	Top-k，k=2	Top-p，p=0.8
苹果	0.4	0.4	保留	保留
香蕉	0.3	0.7	保留	保留
桌子	0.2	0.9	排除	保留
篮球	0.1	1.0	排除	排除

Top-p保留累计概率达到阈值的最小候选集合。

保留后重新归一化。p=0.8时保留前三项，其概率变为4/9、3/9、2/9。

PyTorch实现：Top-p采样

```
def sample_top_p(logits, top_p=0.8):
    assert 0 < top_p <= 1
    probs, ids = logits.softmax(-1).sort(descending=True)
    cumulative_before = probs.cumsum(-1) - probs
    keep = cumulative_before < top_p
    probs = probs * keep
    probs = probs / probs.sum()
    picked = torch.multinomial(probs, 1)
    return ids[picked]
next_id = sample_top_p(z, top_p=0.8)
```

本例只处理一个位置的一维logits。批量生成还需分别维护各序列的结束状态。

Beam Search：保留多个候选前缀

候选序列	第一步概率	第二步条件概率	序列概率
A a	0.60	0.51	0.306
A b	0.60	0.49	0.294
B c	0.40	0.99	0.396
B d	0.40	0.01	0.004

贪心选择 A a。束宽2在本例中能找到 B c。

所有候选长度相同。有限束宽的剪枝搜索不保证全局最优。

生成实现中的常见问题

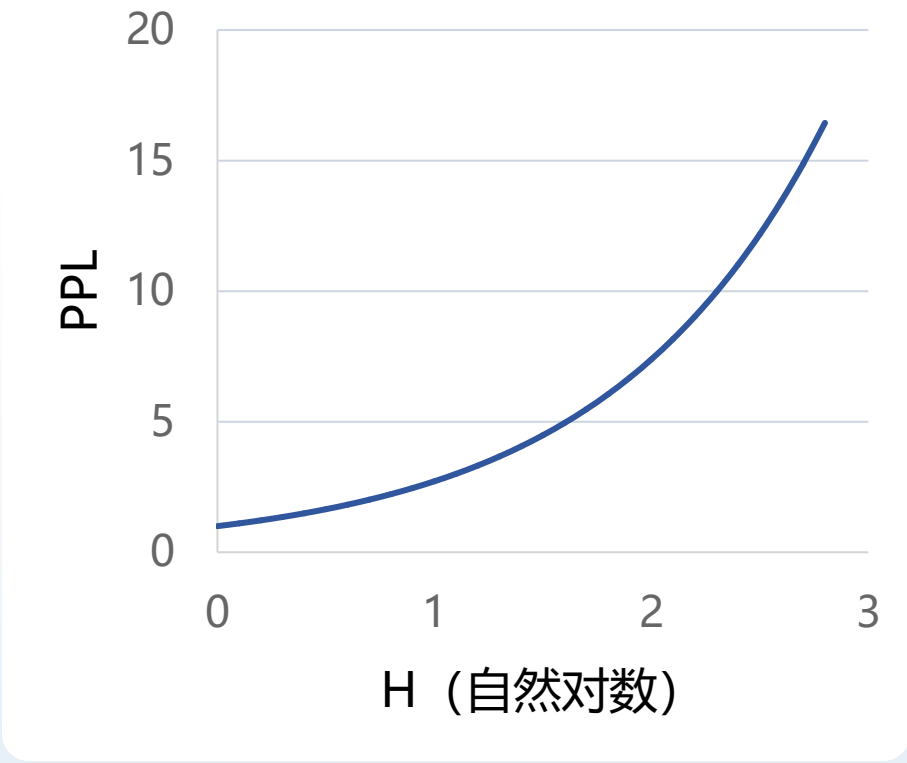
上下文截取	x[:, -K:] 保留最近K个词，K为输入窗口上限
短输入与Padding	单条短输入可直接计算，因果Mask不替代Padding Mask
EOS与批量生成	单样本可用.item()，批量需为每条序列维护结束标记
推断状态	model.eval()与inference_mode()分别控制模块行为和梯度

采样温度要求大于0。贪心解码直接使用argmax。

困惑度与平均负对数似然

$$H = -(1/N)\ \Sigma_i \ln p_i$$
$$PPL = \exp(H)$$

p_i	模型分配给真实下一词的概率
H	有效目标的平均负对数似然
PPL	在同一评价条件下越低越好



N统计有效目标。若用$\log_2$计算H，应使用2^H，而非exp(H)。

课堂手算：三个目标词元的困惑度

目标位置	真实目标概率 p_i	$-\log_2 p_i$
1	1 / 2	1
2	1 / 4	2
3	1 / 8	3

平均损失 $H = (1 + 2 + 3) / 3 = 2$ bits
$PPL = 2^2 = 4$

使用自然对数时：$H = \ln 4 \approx 1.386$，$PPL = \exp(1.386) \approx 4$。

PPL统计：先累计损失，再统一除以词元数

批次	有效目标数	平均损失
A	3	ln 2
B	1	ln 16

整体 PPL = exp[(3 ln 2 + ln 16) / 4] ≈ 3.36

```
nll_sum += F.cross_entropy(logits.reshape(-1, 12),
    targets.reshape(-1), ignore_index=0, reduction="sum").item()
n_tokens += targets.ne(0).sum().item()
# 所有批次累计完成后
ppl = math.exp(nll_sum / n_tokens)
```

遍历前令nll_sum = n_tokens = 0。用有效词元数加权，避免批次长度造成偏差。

困惑度可比性的条件

相同测试文本	数据领域、清洗与文档边界一致
相同词元化	词、字、子词的PPL不能直接横向比较
相同上下文口径	窗口长度、步长和目标计分规则一致
相同目标集合	EOS规则一致，Padding和重叠目标不重复计分

经典下一词PPL也不能直接套用于BERT式掩码语言模型。

概率评分与实际应用质量

评价方式	它衡量什么	仍需另外检查
困惑度 PPL	对真实文本的下一词预测能力	能否满足具体任务要求
生成文本分析	流畅性、重复、词汇多样性	内容事实是否可靠
外部任务评价	翻译、问答等任务中的实际效果	评价集是否覆盖真实场景

“读起来流畅”和“内容正确”需要分别判断。

本章学习概率建模。指令微调、偏好对齐与应用评价在后续章节展开。

自回归生成的效率：缓存与推测解码

KV缓存

保存已读位置的Key、Value
生成下一词时复用这些历史计算

投机解码

草稿模型先提出多个词元
目标模型批量验证，接受或校正

评价成本

同时看吞吐量、延迟、显存与输出质量

经典推测解码可保持目标模型的采样分布，实际提速依赖草稿接受率等条件。

DeepSeek-V3（2024/2025）：为下一词训练增加辅助目标

模块	训练时使用的信息	预测目标
主模型	上文“我 喜欢”	读书
附加预测模块	主干状态与“读书”的词向量	<eos>

更多监督信号帮助学习表示，也可为推测解码提供草稿。

这是V3附加预测链的教学示例。训练目标、推断策略与验证机制需要区分。

研究拓展：扩散语言模型

LLaDA（2025）：通过多轮恢复掩码生成文本。

阶段	位置1	位置2	位置3	位置4
初始化	[MASK]	[MASK]	[MASK]	[MASK]
一轮恢复	[MASK]	喜欢	[MASK]	<eos>
继续恢复	我	喜欢	读书	<eos>

MRP（2026）：预测相邻去噪步骤的logits修正量，减少重复计算。

表格为教学示意。并行程度、迭代轮数、速度与质量需要共同评价。

本章小结

语言模型	估计下一词的条件概率，并组合为序列概率
模型结构	计数、固定窗口、递推状态、因果注意力
文本生成	根据分布解码，直到EOS或长度上限
性能评价	统一口径累计PPL，再检查实际应用效果

谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR